

HAFTA 3 RAPORU

```
(kali@kali)-[~]
$ sslscan 127.0.0.1:8443
Version: 2.1.5
OpenSSL 3.6.2 7 Apr 2026

Connected to 127.0.0.1

Testing SSL server 127.0.0.1 on port 8443 using SNI name 127.0.0.1

SSL/TLS Protocols:
SSLv2      disabled
SSLv3      disabled
TLSv1.0    disabled
TLSv1.1    disabled
TLSv1.2    disabled
TLSv1.3    enabled

TLS Fallback SCSV:
Server supports TLS Fallback SCSV

TLS renegotiation:
Session renegotiation not supported

TLS Compression:
Compression disabled

Heartbleed:
TLSv1.3 not vulnerable to heartbleed

Supported Server Cipher(s):
Preferred TLSv1.3 256 bits TLS_AES_256_GCM_SHA384
Accepted TLSv1.3 256 bits TLS_CHACHA20_POLY1305_SHA256
Accepted TLSv1.3 128 bits TLS_AES_128_GCM_SHA256

Server Key Exchange Group(s):
TLSv1.3 128 bits secp256r1 (NIST P-256)
TLSv1.3 192 bits secp384r1 (NIST P-384)
TLSv1.3 260 bits secp521r1 (NIST P-521)
TLSv1.3 128 bits x25519
TLSv1.3 224 bits x448
TLSv1.3 112 bits ffdhe2048
TLSv1.3 128 bits ffdhe3072

SSL Certificate:
Signature Algorithm: sha256WithRSAEncryption
RSA Key Strength: 2048

Subject: lab.local
Altnames: DNS:lab.local, DNS:localhost, IP Address:127.0.0.1
Issuer: Siber Güvenlik Ders Kök CA - 232923067

Not valid before: Oct 5 15:38:07 2026 GMT
Not valid after: Jan 8 15:38:07 2029 GMT

(kali@kali)-[~]
$
```

```
(kali@kali)-[~]  
$ sslscan 127.0.0.1:8445  
Version: 2.1.5  
OpenSSL 3.6.2 7 Apr 2026
```

```
Connected to 127.0.0.1
```

```
Testing SSL server 127.0.0.1 on port 8445 using SNI name 127.0.0.1
```

SSL/TLS Protocols:

```
SSLv2      disabled  
SSLv3      disabled  
TLSv1.0    enabled  
TLSv1.1    enabled  
TLSv1.2    enabled  
TLSv1.3    enabled
```

TLS Fallback SCSV:

```
Server supports TLS Fallback SCSV
```

TLS renegotiation:

```
Secure session renegotiation supported
```

TLS Compression:

```
Compression disabled
```

Heartbleed:

```
TLSv1.3 not vulnerable to heartbleed  
TLSv1.2 not vulnerable to heartbleed  
TLSv1.1 not vulnerable to heartbleed  
TLSv1.0 not vulnerable to heartbleed
```

Supported Server Cipher(s):

```
Preferred TLSv1.3 256 bits TLS_AES_256_GCM_SHA384
```

```
Accepted TLSv1.3 256 bits TLS_CHACHA20_POLY1305_SHA256  
Accepted TLSv1.3 128 bits TLS_AES_128_GCM_SHA256  
Preferred TLSv1.2 256 bits ECDHE-RSA-AES256-GCM-SHA384 Curve 25519 DHE 253  
Accepted TLSv1.2 256 bits ECDHE-RSA-CHACHA20-POLY1305 Curve 25519 DHE 253  
Accepted TLSv1.2 256 bits ECDHE-ARIA256-GCM-SHA384 Curve 25519 DHE 253  
Accepted TLSv1.2 128 bits ECDHE-RSA-AES128-GCM-SHA256 Curve 25519 DHE 253  
Accepted TLSv1.2 128 bits ECDHE-ARIA128-GCM-SHA256 Curve 25519 DHE 253  
Accepted TLSv1.2 256 bits ECDHE-RSA-AES256-SHA384 Curve 25519 DHE 253  
Accepted TLSv1.2 256 bits ECDHE-RSA-CAMELLIA256-SHA384 Curve 25519 DHE 253  
Accepted TLSv1.2 128 bits ECDHE-RSA-AES128-SHA256 Curve 25519 DHE 253  
Accepted TLSv1.2 128 bits ECDHE-RSA-CAMELLIA128-SHA256 Curve 25519 DHE 253  
Accepted TLSv1.2 256 bits ECDHE-RSA-AES256-SHA Curve 25519 DHE 253  
Accepted TLSv1.2 256 bits AECDH-AES256-SHA Curve 25519 DHE 253  
Accepted TLSv1.2 128 bits ECDHE-RSA-AES128-SHA Curve 25519 DHE 253  
Accepted TLSv1.2 128 bits AECDH-AES128-SHA Curve 25519 DHE 253  
Accepted TLSv1.2 256 bits AES256-GCM-SHA384  
Accepted TLSv1.2 256 bits AES256-CCM  
Accepted TLSv1.2 256 bits ARIA256-GCM-SHA384  
Accepted TLSv1.2 128 bits AES128-GCM-SHA256  
Accepted TLSv1.2 128 bits AES128-CCM  
Accepted TLSv1.2 128 bits ARIA128-GCM-SHA256  
Accepted TLSv1.2 64 bits AES256-CCM8  
Accepted TLSv1.2 64 bits AES128-CCM8  
Accepted TLSv1.2 256 bits AES256-SHA256  
Accepted TLSv1.2 256 bits CAMELLIA256-SHA256  
Accepted TLSv1.2 128 bits AES128-SHA256  
Accepted TLSv1.2 128 bits CAMELLIA128-SHA256  
Accepted TLSv1.2 256 bits AES256-SHA  
Accepted TLSv1.2 256 bits CAMELLIA256-SHA  
Accepted TLSv1.2 128 bits AES128-SHA  
Accepted TLSv1.2 128 bits SEED-SHA  
Accepted TLSv1.2 128 bits CAMELLIA128-SHA  
Preferred TLSv1.1 256 bits ECDHE-RSA-AES256-SHA Curve 25519 DHE 253  
Accepted TLSv1.1 256 bits AECDH-AES256-SHA Curve 25519 DHE 253
```

```
Accepted TLSv1.1 128 bits ECDHE-RSA-AES128-SHA Curve 25519 DHE 253
Accepted TLSv1.1 128 bits AECDH-AES128-SHA Curve 25519 DHE 253
Accepted TLSv1.1 256 bits AES256-SHA
Accepted TLSv1.1 256 bits CAMELLIA256-SHA
Accepted TLSv1.1 128 bits AES128-SHA
Accepted TLSv1.1 128 bits SEED-SHA
Accepted TLSv1.1 128 bits CAMELLIA128-SHA
Preferred TLSv1.0 256 bits ECDHE-RSA-AES256-SHA Curve 25519 DHE 253
Accepted TLSv1.0 256 bits AECDH-AES256-SHA Curve 25519 DHE 253
Accepted TLSv1.0 128 bits ECDHE-RSA-AES128-SHA Curve 25519 DHE 253
Accepted TLSv1.0 128 bits AECDH-AES128-SHA Curve 25519 DHE 253
Accepted TLSv1.0 256 bits AES256-SHA
Accepted TLSv1.0 256 bits CAMELLIA256-SHA
Accepted TLSv1.0 128 bits AES128-SHA
Accepted TLSv1.0 128 bits SEED-SHA
Accepted TLSv1.0 128 bits CAMELLIA128-SHA
```

Server Key Exchange Group(s):

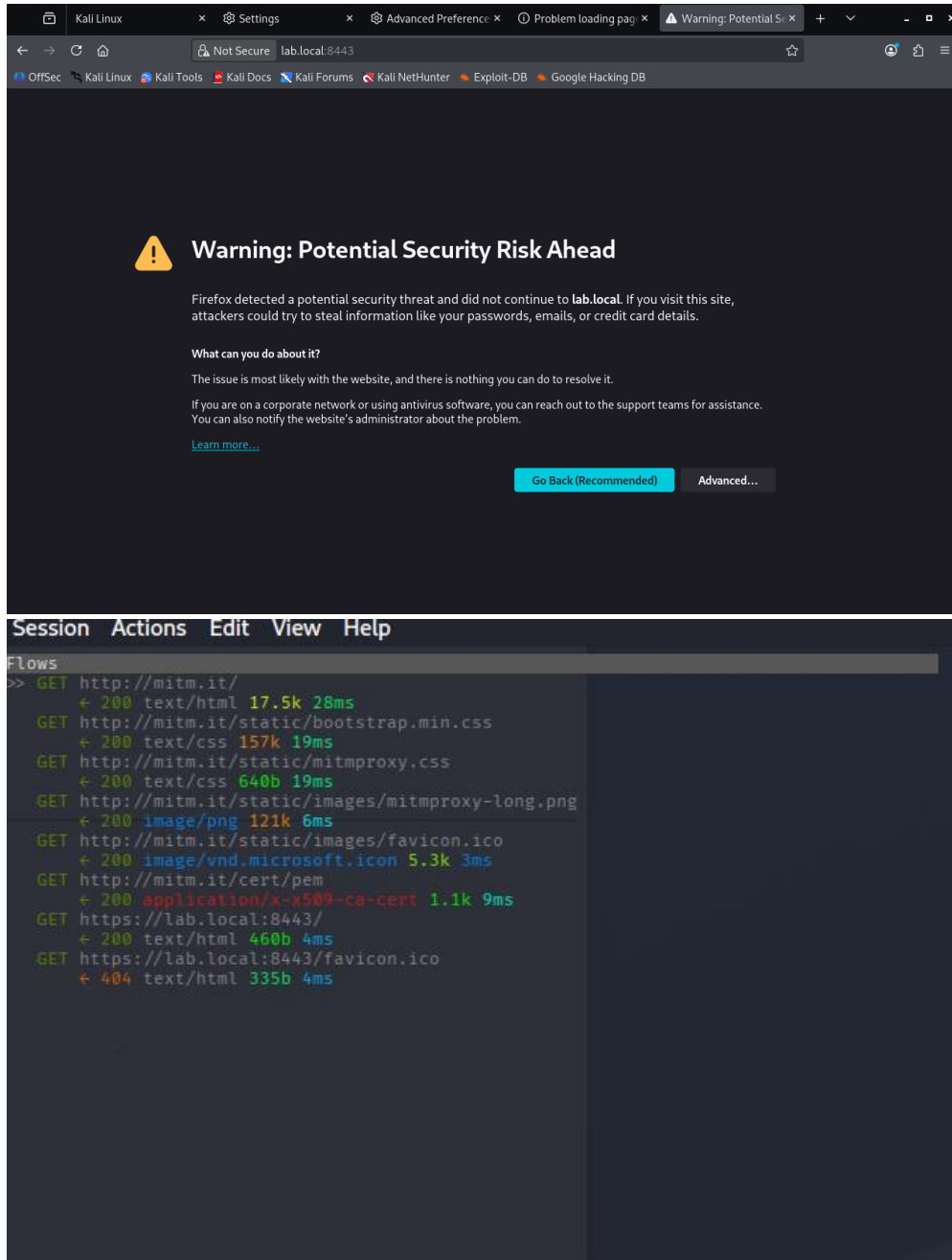
```
TLSv1.3 128 bits secp256r1 (NIST P-256)
TLSv1.3 192 bits secp384r1 (NIST P-384)
TLSv1.3 260 bits secp521r1 (NIST P-521)
TLSv1.3 128 bits x25519
TLSv1.3 224 bits x448
TLSv1.3 112 bits ffdhe2048
TLSv1.3 128 bits ffdhe3072
TLSv1.2 128 bits secp256r1 (NIST P-256)
TLSv1.2 192 bits secp384r1 (NIST P-384)
TLSv1.2 260 bits secp521r1 (NIST P-521)
TLSv1.2 128 bits x25519
TLSv1.2 224 bits x448
```

SSL Certificate:

```
Signature Algorithm: sha256WithRSAEncryption
RSA Key Strength: 2048
```

```
Subject: lab.local
AltNames: DNS:lab.local, DNS:localhost, IP Address:127.0.0.1
Issuer: Siber Güvenlik Ders Kök CA - 232923067
```

```
Not valid before: Oct 5 15:38:07 2026 GMT
Not valid after: Jan 8 15:38:07 2029 GMT
```



Soru 6.1

Soru: sslscan çıktısında güçlü sunucunuz hangi TLS sürümünü ve hangi şifre takımını tercih etti? Bu değerler ana laboratuvardaki gorev2_istemi.py çıktısıyla uyuyor mu?

Cevap: Güçlü sunucu taramasında yalnızca TLSv1.3 protokolü aktif (enabled) durumdadır; SSLv2, SSLv3, TLS 1.0, 1.1 ve 1.2 tamamen devre dışı bırakılmıştır. Sunucunun tercih ettiği şifre takımı TLS_AES_256_GCM_SHA384 (256-bit) olarak tespit

edilmiştir. Kabul edilen diğer şifreleyiciler ise TLS_CHACHA20_POLY1305_SHA256 ve TLS_AES_128_GCM_SHA256'dır. Bu parametreler, ana laboratuvarında doğrudan sunucuya bağlanan `gorev2_istemci.py` betiğinin kurduğu TLS 1.3 el sıkışması ve şifreleme çıktılarıyla birebir örtüşmektedir.

Soru 6.2

Soru: Güçlü ve zayıf sunucu taramaları arasındaki fark neydi? Bir sistem yöneticisi olsanız, bir sunucuda hangi TLS sürümlerini kapatırdınız ve neden?

Cevap: Güçlü sunucu yalnızca TLS 1.3 ve modern AEAD tabanlı şifre takımlarını desteklerken; zayıf sunucu geriye dönük olarak TLSv1.0, TLSv1.1 ve TLSv1.2 protokollerinin tamamına izin vermiştir. Ayrıca zayıf sunucu kimlik doğrulaması içermeyen (AECDH), zayıf blok boyutuna sahip (AES256-CCM8 64-bit) ve CBC modlu güvensiz şifre takımlarını da kabul etmiştir.

Bir sistem yöneticisi olsaydım SSLv2, SSLv3, TLS 1.0 ve TLS 1.1 sürümlerini istisnasız kapatırdım. Çünkü bu eski sürümler POODLE, BEAST, CRIME gibi yapısal saldırılara açıktır ve modern şifreleme standartlarını karşılamaz. Sunucuda yalnızca TLS 1.3 ve yalnızca güvenli şifre takımlarıyla sınırlandırılmış TLS 1.2 protokollerini açık tutardım.

Soru 6.3

Soru: mitmproxy'nin CA'sını eklemeyen önce Firefox neden uyarı verdi, ekledikten sonra neden vermedi? Bu, "bir kök CA'ya güvenmek" kararını nasıl açıklıyor?

Cevap: İlk aşamada mitmproxy araya girip istemciye anlık olarak kendi ürettiği sahte bir `lab.local` sertifikası sunmuştur. Ancak bu sertifikayı imzalayan kök CA makamı Firefox'un güvenilen sertifika deposunda (Trust Store) bulunmadığından güven zinciri doğrulanamamış ve tarayıcı kullanıcıyı uyararak bağlantıyı engellemiştir.

mitmproxy'nin kök sertifikası yetkili makamlara eklendiğinde ise tarayıcı bu kökü meşru bir otorite olarak tanımış, dolayısıyla onun imzaladığı sahte sertifikayı da sorgusuz sualsiz geçerli kabul etmiştir. Bu durum, internet güvenliğinin ve PKI hiyerarşisinin temelde yerel depoda tanımlı kök CA'lara duyulan mutlak güvene dayandığını; bu depoya yabancı bir kök eklendiğinde tüm HTTPS şifrelemesinin anlamsızlaşarak trafiğin üçüncü taraflarca tamamen okunabilir hale geleceğini açıklar.

Soru 6.4

Soru: Bu saldırının çalışması için saldırganın neye sahip olması gerekiyordu? Gerçek bir web sitesinde (örneğin bankanızda) bir saldırgan bunu neden kolayca yapamaz?

Cevap: Saldırının başarılı olması için saldırganın hem ağ seviyesinde trafiği kendi üzerinden yönlendirebilmesi (vekil sunucu, ARP zehirlleme, DNS spoofing vb.) hem de kurbanın cihazına/tarayıcısına kendi kök sertifikasını yükletebilmesi gerekirdi.

Gerçek bir web sitesinde (örneğin bir bankada) saldırganın bunu kolayca yapamamasının nedenleri şunlardır:

Sertifika Şeffaflığı (Certificate Transparency - CT): Genel geçerli bir CA tarafından üretilen her sertifika herkese açık, değiştirilemez CT kayıt kütüklerine yazılmak zorundadır. Yetkisiz üretilen sahte sertifikalar alan adı sahipleri ve denetçiler tarafından hemen tespit edilir ve sertifika derhal iptal edilir (Revocation).

CAA (DNS Certification Authority Authorization): Alan adı sahipleri DNS kayıtlarında yalnızca hangi yetkili makamların (CA) kendileri adına sertifika düzenleyebileceğini kısıtlar. Farklı bir CA'dan sertifika talep edildiğinde işlem reddedilir.

Sertifika Sabitleme (Certificate Pinning / HPKP): Bankacılık ve kritik web/mobil uygulamaları sunucunun belirli açık anahtarlarını veya parmak izlerini uygulama içine gömer. Araya giren saldırgan başka bir geçerli CA kullansa bile anahtar eşleşmediği için bağlantı derhal kesilir.

Onay Beyanı:

Laboratuvar çalışmasının ardından Firefox vekil sunucu (proxy) ayarı kapatılmış ("No proxy" seçilmiş), mitmproxy kök CA sertifikası Authorities listesinden silinmiş ve about:config üzerindeki network.proxy.allow_hijacking_localhost değeri tekrar false yapılmıştır.